

Team Management

Partner, agent and internal-team scoping for the Vend88 Agent Portal

Design proposal · 17 September 2026 · Not yet implemented — no code changed

1. Purpose and scope	9. Organisation self-administration
2. The four stakeholders	10. Reassignment and audit
3. Core concept: permission × scope	11. Frontend structure
4. Data model	12. Migration and backfill
5. Visibility matrix	13. Build phases
6. Attribution: where ownership is stamped	14. Decisions taken
7. Enforcement design	15. Assumptions and open risks
8. Revenue and reporting scope	

1. Purpose and scope

Today the portal has two effective tiers: `admin` and `super_admin`, both of which see every record in the system. This design adds the ability to hand the portal to **external partners** — organisations and individual agents who promote and sell the system — and to **internal sales staff** who should only see the clients they personally brought in.

The central requirement: every customer, business and deal is attributed to the person and team who originated it. Those parties see only their own book. Administrators continue to see everything, and additionally see *which team each entity belongs to*.

2. The four stakeholders

Stakeholder	Who they are	What they need to see
Administrator	Vend88 owner, developers, operations	Everything, with a team and owner column on every list, plus the ability to reassign
Internal team member	Business development managers on Vend88 staff	Only the clients they personally brought in
Individual agent	External one-person reseller	Only their own clients
Organisation	External partner company that builds its own sub-team	Owner sees the whole organisation's book; each member sees only their own

The insight that shapes everything below

"Internal vs external" and "restricted vs unrestricted" are *two different axes*. A BDM is internal but restricted; an administrator is internal and unrestricted. Collapsing them into one flag does not work — which is why scope below is expressed independently of who employs the person.

3. Core concept: permission × scope

The existing permission system (`utils/portalAuth.js`) answers one question: *what actions may this user perform?* — `view_businesses`, `manage_quotations`, and so on. Partner access needs a second, independent question: *which records may this user touch?*

These stay separate:

- **Permission** — unchanged, per-module action rights.
- **Scope** — derived from the user's team and visibility setting; applied as a query filter.

An organisation member and an administrator can therefore hold the *same* `view_businesses` permission and run through the *same* code path. Only the filter differs.

Why not just add more permissions

The current `view_own_businesses` vs `view_businesses` split is an attempt to encode scope as permission strings. It does not scale: every permission would need an `_own_` twin, then a `_team_` twin, and no combination can express "the whole organisation but not other organisations". Scope must be data, not a permission name.

4. Data model

4.1 New collection: `team`

```
{
  _id,
  name, // "Acme Retail Solutions" / "Jane Chen"
  kind: 'internal' | 'organisation' | 'individual',
  status: 'active' | 'suspended',
  contact_name, contact_email, contact_phone, abn,
  commission_rate, // reserved for later
  notes,
  created_by, created_at
}
```

Every non-administrator belongs to a team. An individual agent gets a team row of one, and internal BDMs sit in a single `vend88-direct` team. A uniform shape is deliberate — the alternative, a null-team special case, has to be remembered in every query that is written from here on.

4.2 Additions to `portal_user`

```
team_id: String // null only for administrators
visibility: 'self' | 'team' | 'all'
```

One field, `visibility`, produces every stakeholder without a special case anywhere in the code:

Stakeholder	team_id	visibility	Resulting filter
Administrator	null	all	no filter
BDM (internal)	vend88-direct	self	owner_user_id = me
Individual agent	own team row	self	owner_user_id = me
Organisation owner	org team	team	team_id = my team
Organisation member	org team	self	owner_user_id = me

A sales manager sitting above the BDMs, should you want one later, is simply `team_id: vend88-direct,`
`visibility: team`. No new code, no new role.

4.3 Attribution fields on every root entity

Added to `business`, `registration`, `quotation` and `inquiry`:

```
owner_user_id: String // the individual who originated it – always set
team_id:       String // their team at the time – null for an admin's own deal
```

Both are stored, not one derived from the other. A person can move between teams; the record must keep the team it was won under, or historical reporting and commission silently rewrite themselves.

Existing attribution is stored as email strings

The portal currently records `generated_by: actor.email` on registrations and `created_by: user.email` on quotations. That is acceptable for an audit trail but unsafe as a security filter — a single email change would detach a salesperson from their entire book. `owner_user_id` must hold the `portal_user _id`, with the email retained alongside for display only.

4.4 What is deliberately *not* stamped

`shop`, `order` and `transaction` carry no team field. They resolve through `business_id → business.team_id`. One source of truth means a reassignment moves everything at once, rather than leaving stale stamps scattered across high-volume collections.

5. Visibility matrix

What each stakeholder can see and do, once the model above is in place.

Capability	Admin	BDM	Agent	Org owner	Org member
See all businesses	Yes	No	No	No	No
See own clients	Yes	Yes	Yes	Yes	Yes
See colleagues' clients	Yes	No	—	Own org	No
Revenue for clients in scope	Yes	Yes	Yes	Yes	Yes
Generate registration links	Yes	Yes	Yes	Yes	Yes
Approve / provision accounts	Yes	No	No	No	No
Create member logins	Yes	No	No	Own org	No
Reassign a client	Anywhere	No	No	Within org	No
See which team owns an entity	Yes	N/A	N/A	N/A	N/A

Strict self scope

Where visibility is `self`, it means self without exception — there is no secondary-assignee or shared-client concept. Holiday cover and staff turnover are handled by reassignment (an administrator, or an org owner within their own org), which is recorded in the audit log. The benefit is that "who owns this deal" always has exactly one answer, which matters when commission is calculated, and that a partner can be shown a simple, provable statement about how their data is walled off.

6. Attribution: where ownership is stamped

Entity	Stamped at
<code>registration</code>	the moment the link is generated — alongside the existing <code>generated_by</code>
<code>business</code>	copied from the registration inside <code>provisionBusinessAccount</code>
<code>quotation</code>	on create, from the authenticated caller
<code>inquiry</code>	on create, or on assignment for inbound public-website leads

Stamping at *link generation* rather than at approval is the important detail: attribution is settled before the customer has even filled in the form, so there is never a dispute about who brought a deal that took weeks to close.

7. Enforcement design

The design is not the risky part — the enforcement pass is. `routes/portal.js` alone carries roughly thirty endpoints, several of which currently run a bare `Business.find({})` or `User.find({})`. Every one that is missed is a partner reading another partner's book.

7.1 Make the unscoped call site look wrong

`requirePortalPermission` should return `{ user, scope }` rather than `user`. Every list query then reads `Model.find({ ...scope, ...query })`, and an endpoint that ignores scope becomes visibly different from the ones around it — instead of looking exactly like today's correct code, which is what allows a gap to survive review.

7.2 Three classes of endpoint, three treatments

Class	Treatment
List endpoints <code>businesses, customers, shops, registrations, quotations, inquiries</code>	Merge the scope filter into the query.
Detail and mutation endpoints <code>/businesses/:id, /shops/:id, owner-credentials</code>	Fetch by id, then check the record's owner/team against the caller's scope. Respond 404, not 403 — a 403 confirms the record exists, which is itself a disclosure.
Aggregation endpoints <code>transactions, revenue, growth, pipeline</code>	Resolve the caller's shop ids and prepend a <code>\$match</code> to each pipeline.

The aggregations are the easiest to overlook

The four summary endpoints currently aggregate across *all* orders with no business filter whatsoever. Because there is no `find({})` to search for, they will not show up in a grep for unscoped queries — they have to be found deliberately.

8. Revenue and reporting scope

Everyone sees revenue within their own scope: a member for their own clients, an owner across their organisation. Revenue is treated as just another field behind the same filter, with no second gate — a separate rule would be one more thing to get wrong.

The resolution path, and why it is cheap:

```
team / user scope
→ business ids  (business.team_id or business.owner_user_id)
→ shop ids      (shop.business_id ∈ business ids)
→ { $match: { shop_id: { $in: shopIds }, time: { ... } } }
```

The `order` collection already carries `shop_id` and has a `{ shop_id, time }` compound index, so the scoped aggregation rides an existing index rather than forcing a collection scan.

9. Organisation self-administration

Organisation owners create, suspend and remove their own member logins. This is what lets partners grow without Vend88 staff in the loop for every hire — and it is the only genuine privilege-escalation surface in the design, because it means an external party writes `portal_user` rows.

Hard server-side caps

Constraint	Rule
Team assignment	<code>team_id</code> forced to the caller's own — never read from the request body
Role ceiling	capped at or below the caller's own; <code>admin</code> and <code>super_admin</code> unreachable
Visibility ceiling	may grant <code>self</code> , and <code>team</code> to a deputy inside their own org; never <code>all</code>
Suspension cascade	suspending a team revokes its members' access even while their own status reads <code>active</code>
Provisioning	link generation is self-serve; approval and account provisioning stay administrator-only

Build this as its own guarded function

It should not reuse the existing `/admin/create` endpoint with extra branches. That endpoint assumes a fully trusted caller throughout; introducing a semi-trusted one into it is precisely how a gap gets in. A separate, independently tested path is the safer shape.

10. Reassignment and audit

- **Administrators** may move any client to any person or team.
- **Organisation owners** may reallocate between their own members, but never across the team boundary — in or out.
- Every reassignment is written to the existing `admin_audit_log`.

This is worth building early rather than retrofiting. Partners hand deals over, staff leave, and commission disputes are real; without it the fallback is editing MongoDB by hand, with no record of who changed what.

11. Frontend structure

One portal, one module registry. `config/adminModules.ts` already drives the sidebar, the dashboard cards and the permission editor from a single list. A team user logging in receives a smaller permission set and scoped data through the same screens. A parallel partner application would have to be kept in step forever, and would drift.

- **Team Management** (administrator-only) — list and create teams, manage member logins, view each team's attributed businesses and pipeline.
- **Every admin list** — Businesses, Customers, Registrations, Quotations — gains a *Team* column and a *Team* filter. This is what delivers "see everything, including which entity belongs to which team".

- **Partner-facing views** — the same components, rendering whatever the scoped API returns. No separate templates.

12. Migration and backfill

Existing data	Treatment
All current businesses and deals	<code>team_id: null</code> — house accounts, owned by Vend88
Current admin / super_admin users	<code>team_id: null</code> , <code>visibility: 'all'</code> — unchanged behaviour
Historic <code>generated_by</code> / <code>created_by</code> emails	backfill <code>owner_user_id</code> by matching email to <code>portal_user</code> ; leave null where no match, retain the email for display

The defaults are safe in both directions: an unmigrated *user* sees everything only if they were already an administrator, and an unmigrated *record* belongs to Vend88.

13. Build phases

Phase	Work	Risk / notes
1	<code>Team</code> model; <code>team_id</code> , <code>owner_user_id</code> , <code>visibility</code> fields; <code>utils/teamScope.js</code> ; <code>requirePortalPermission</code> returns scope	No behaviour change on its own. Makes every later unscoped query visible rather than silent.
2	Attribution stamping at registration generation, provisioning, quotation and inquiry create	Low risk; additive.
3	Scope enforcement across list, detail and aggregation endpoints	The critical pass. Every missed endpoint is a data leak. Needs a written endpoint inventory and a test per class.
4	Team Management UI; team and owner columns and filters on admin lists	Administrator-facing only.
5	Organisation self-administration with the escalation caps	The security-sensitive surface. Independent tests.
6	Partner-facing portal views; reassignment UI and audit	Ship reassignment before the first real partner is onboarded.

14. Decisions taken

Question	Decision
What can a team see about their businesses?	Pipeline and revenue for businesses in their scope
How much can a team do unaided?	Self-serve registration links ; approval and provisioning remain with administrators
Who manages an organisation's member logins?	The organisation owner , within their own org, under hard server-side caps
Who can reassign a client?	Administrators anywhere; org owners within their own org. Both audited
Can colleagues see each other's clients for cover?	No — strict self scope. Cover is handled by audited reassignment
Does revenue visibility reach org members?	Yes — everyone sees revenue within their own scope

15. Assumptions and open risks

Assumptions — correct these if wrong

1. **Administrators can personally own clients.** An admin who also sells carries `owner_user_id` with `team_id: null`, so they are not forced into a fictitious team. They continue to see everything.
2. **Attribution survives a person leaving.** Suspending a user does not clear `owner_user_id`; clients stay attributed until an administrator reassigns them, so history and commission do not evaporate.

Risks to watch

- **Enforcement coverage.** The single largest risk in the project. Phase 3 needs a complete endpoint inventory, not an ad-hoc sweep.
- **Suspended teams with active members.** The team status check must live inside scope resolution, not only in the user status check, or a suspended partner retains revenue access.
- **Escalation through self-administration.** Every cap in section 9 must be enforced server-side; UI-level restriction is not a control.
- **Commission is not yet modelled.** `commission_rate` is reserved on the team record, but no calculation exists. Attribution being correct from day one is what keeps that option open.
- **End-customer privacy.** Partners will see their customers' trading revenue. Worth confirming this sits correctly with the terms those customers agreed to.